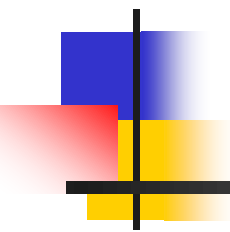
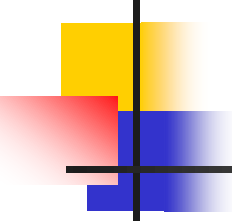


07. Exception Handling

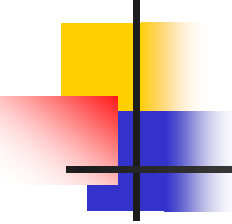


- ◆ Exception Delegation
- ◆ *'try', 'catch', 'throw', 'throws'* and *'finally'* clauses
- ◆ Creating user defined exceptions



Exception

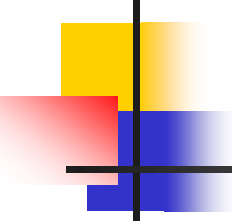
- An exception is an abnormal condition that arises in a code sequence at run time.
- In other words, exception is a run-time error.



Exception

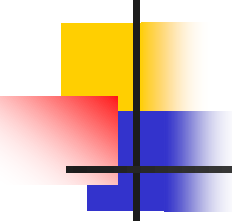
Exceptions can occur when:

- We try to open a non-existing file.
- We have a Disturbed network connection.
- Operands being manipulated are out of prescribed ranges.
- Class file missing which was supposed to be loaded.
- We read beyond array limits.



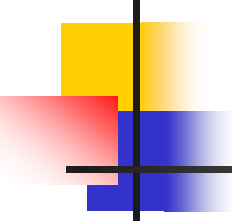
Exception

- An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.
- The exception object contains information about the exception, including its type and the state of the program when the error occurred.
- Helpful in separating the execution code from the error handler.



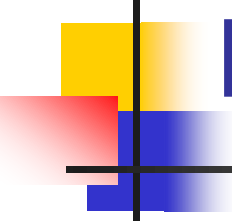
Exception

- The Java programming language provides a mechanism known as exceptions to help programs report and handle errors.
- When an error occurs, the program **throws** an exception.
- Normal flow of the program is interrupted
- The runtime attempts to find an exception handler.
- This is actually what makes Java robust.



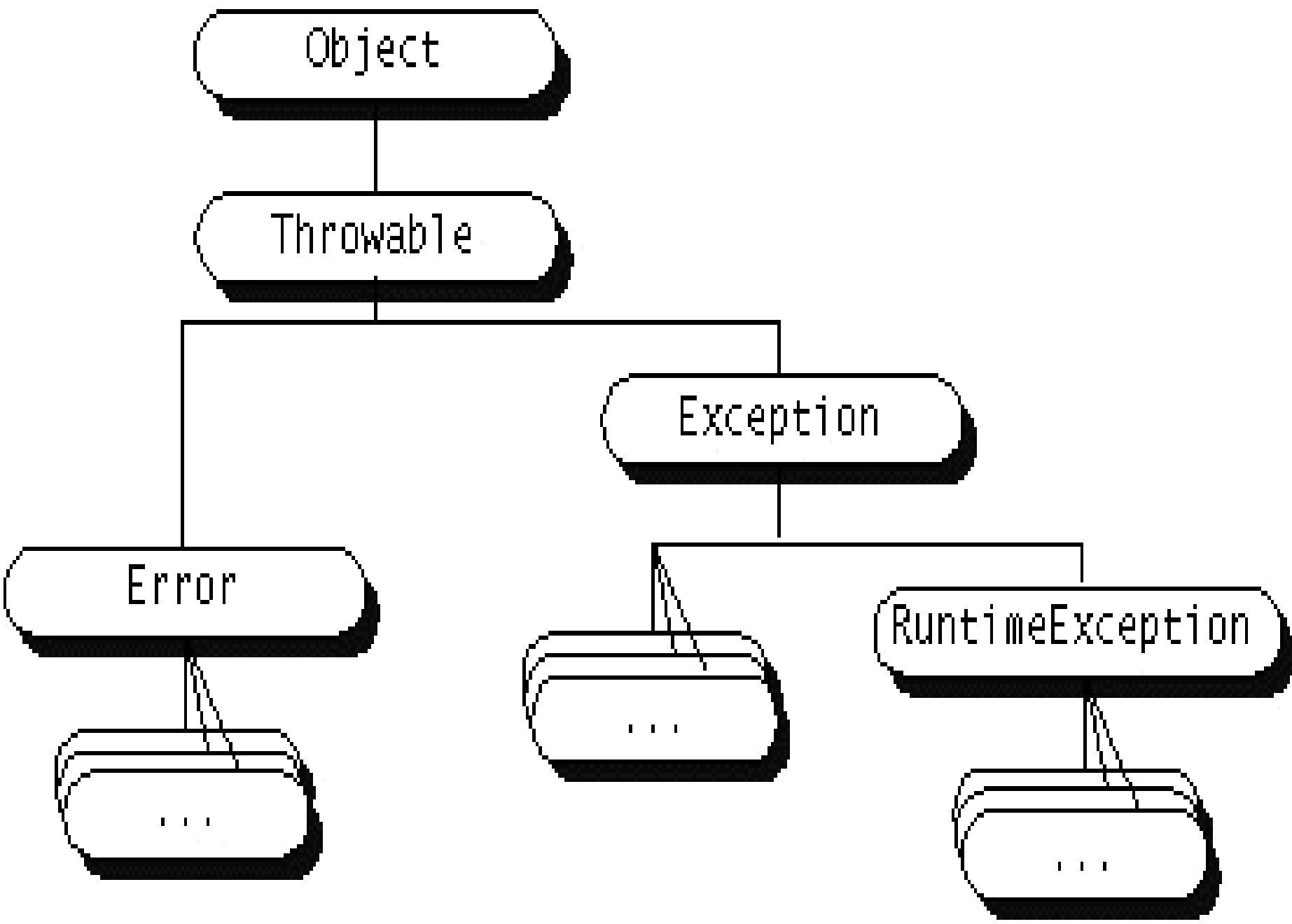
Exceptions

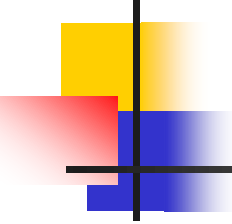
- Three statements play a part in handling exceptions: try – catch – finally
- Java enforces exceptions to be handled by the programmer. If not, compile error occurs.



Exception-handling block

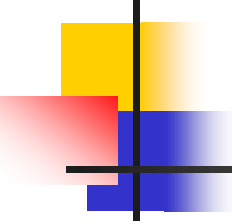
```
try {  
    // block of code need to be monitored for errors.  
}  
catch(ExceptionType1 exOb) {  
    // exception handler for ExceptionType1  
}  
catch(ExceptionType2 exOb) {  
    // exception handler for ExceptionType2  
}  
finally {  
    // block of code to be executed before try block ends.  
}
```





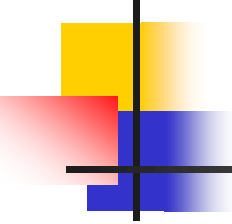
Exception Hierarchy

- We have two branches from the *Throwable* class. One called the **Error**, and the other called **Exception**.
- The **Error** class defines exceptions or problems that are not expected to be caught under normal circumstances by our program.
- Eg: memory error, hardware error, or JVM error etc.



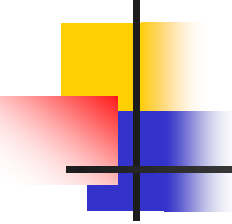
Exception Hierarchy

- **Exception** class, which represent exceptions which can be handled by our program, and our program can recover from these exceptions using the try-catch- finally block.
- **RuntimeException** is one subclass of the Exception class. This cannot be trapped at compile time.
- Example: **DivideByZeroException** or **NullPointerException**.



Exceptions and Errors

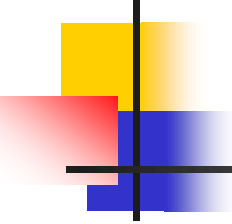
- Exceptions are situations within the control of an application, that it should try to handle
- Errors indicate serious problems and abnormal conditions that most applications should not try to handle



Uncaught Exceptions

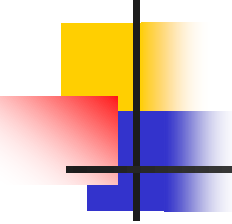
```
class ExceptionTest {  
    public static void main(String args[]) {  
        int d = 0;  
        int a = 42/d;  
    }  
}
```

Exception in thread "main" java.lang.ArithmeticException: / by zero
at Test.main(Test.java:4)



Using *try* and *catch*

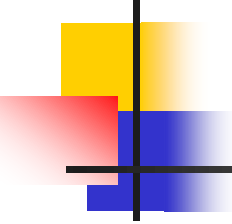
```
class ExceptionTest {  
    public static void main(String args[]) {  
        int d, a;  
        try {  
            d = 0;  
            a = 42/d;  
            System.out.println("This will not be printed");  
        } catch(ArithmeticException e) {  
            System.out.println("Division by zero");  
        }  
        System.out.println("After catch statement.");  
    }  
}
```



Multiple *catch* Clauses

- In some cases, more than one exception could be raised by a single piece of code. To handle this we can specify two or more catch clauses.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.

```
class ExceptionTest {  
    public static void main(String args[]) {  
        try {  
            int a = args.length;  
            System.out.println("a = " + a);  
            int b = 42/a;  
            int c[] = {1};  
            c[42] = 99;  
        } catch(ArithmeticException e) {  
            System.out.println("Division by zero: " + e);  
        } catch(ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index OOB: " + e);  
        }  
  
        System.out.println("After try/catch blocks.");  
    }  
}
```



Output

```
D:\Java Test>java ExceptionTest
```

```
a = 0
```

```
Division by zero: java.lang.ArithmeticException: / by zero
```

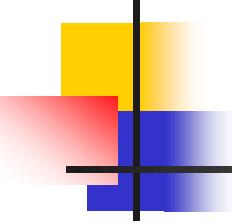
```
After try/catch blocks.
```

```
D:\Java Test>java ExceptionTest something
```

```
a = 1
```

```
Array Index OOB: java.lang.ArrayIndexOutOfBoundsException: 42
```

```
After try/catch blocks.
```



throw Clause

- It is possible for any program to throw an exception explicitly, using the ***throw*** statement. The general form of throw is shown here:

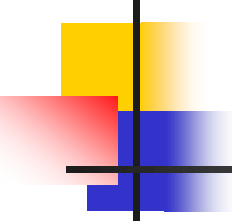
***throw** ThrowableInstance;*

- *ThrowableInstance* must be an object of type *Throwable* or a subclass of *Throwable*.

```
class Test {  
    static void demoproc() {  
        try {  
            throw new NullPointerException("Demo");  
        } catch(NullPointerException e) {  
            System.out.println("Caught inside Demoproc.");  
            throw e;  
        }  
    }  
  
    public static void main(String args[]) {  
        try {  
            demoproc();  
        } catch(NullPointerException e) {  
            System.out.println("Recaught: " +e);  
        }  
    }  
}
```

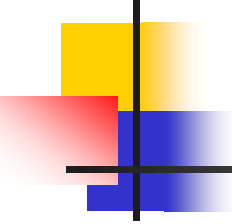
Caught inside Demoproc.

Recaught: java.lang.NullPointerException: Demo



throws Clause

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception. This can be achieved by including *throws* clause in the method's declaration.



throws Clause

The general form of throws is shown here:

```
type methodName(parameter-list) throws  
exception-  
list {  
    // body of method  
}
```

// This program contains an error and will not compile.

```
class ThrowsTest {  
    static void throwOne() {  
        System.out.println("Inside throwOne.");  
        throw new IllegalAccessException("Demo");  
    }  
  
    public static void main(String args[]) {  
        throwOne();  
    }  
}
```

D:\Java Test>javac Test.java

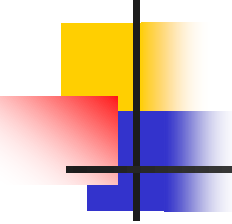
Test.java:5: unreported exception java.lang.IllegalAccessException; must be caught or declared to be thrown

```
        throw new IllegalAccessException("Demo");
```

^

// This program is now correct.

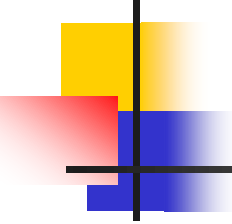
```
class ThrowsTest {  
    static void throwOne() throws IllegalAccessException {  
        System.out.println("Inside throwOne.");  
        throw new IllegalAccessException("Demo");  
    }  
  
    public static void main(String args[]) {  
        try {  
            throwOne();  
        } catch (IllegalAccessException e) {  
            System.out.println("Caught " + e);  
        }  
    }  
}
```



Output

Inside throwOne.

Caught `java.lang.IllegalAccessException`: Demo



finally Clause

- The *finally* statement is associated with a *try* statement and identifies a block of statements that are executed regardless of whether or not an exception occurs within the *try* block.
- Defines the code that is always executed.
- In the normal execution it is executed after the *try* block.
- When an exception, it is executed after the *catch* or before propagation as the case may be.

```
class Test {  
    public static void main(String args[]) {  
        try {  
            int a = args.length;  
            System.out.println("a = " +a + " Inside try");  
            int b = 42/a;  
        } catch(ArithmeticException e) {  
            System.out.println("Inside Catch: " +e);  
        } finally {  
            System.out.println("Inside finally block");  
        }  
        System.out.println("After try/catch/finally blocks.");  
    }  
}
```

Outputs

// With exception

D:\Java Test>java Test

a = 0 Inside try

Inside Catch: java.lang.ArithmeticException: / by zero

Inside finally block

After try/catch/finally blocks.

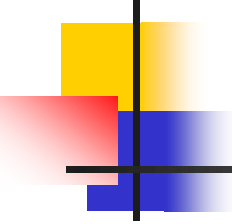
// Normal Execution

D:\Java Test>java Test something

a = 1 Inside try

Inside finally block

After try/catch/finally blocks.

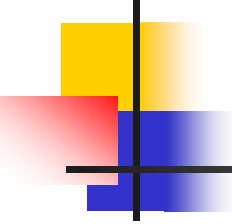


User Defined Exception

- We can create our own exception objects, which can be thrown using the *throw* keyword.
- Create a class which extends the `Exception` class. This is our user-defined exception class, that we want to throw.
- Create an instance of the user-defined exception class.
- Use the throw keyword to throw the instance created.
- Catch it accordingly.

```
class MyException extends Exception {  
    private int detail;  
  
    MyException(int a) {  
        detail = a;  
    }  
  
    public String toString() {  
        return "MyException[" + detail + "];"  
    }  
}
```

```
class ExceptionDemo {  
    static void compute(int a) throws MyException {  
        System.out.println("Called compute(" + a + ")");  
        if (a > 10)  
            throw new MyException(a);  
        System.out.println("Normal Exit");  
    }  
  
    public static void main(String args[]) {  
        try {  
            compute(1);  
            compute(20);  
        } catch(MyException e) {  
            System.out.println("Caught " + e);  
        }  
    }  
}
```



Output

Called compute(1)

Normal Exit

Called compute(20)

Caught MyException[20]